

Analisi Partite Serie A

Componenti del gruppo

- Domenico Ancler, [MAT. 666360], d.ancler@studenti.uniba.it

Link GitHub: [progetto](#)

A. A. 2024-25

Sommario

Capitolo 0 – Introduzione	3
Capitolo 1 – Dataset	5
Capitolo 2 – Apprendimento Supervisionato	6
Capitolo 3 – Rete Bayesiana	9
Capitolo 4 – Rappresentazione della conoscenza e Prolog	12
Capitolo 5 – Ricerca euristica e schedina rc.....	14
Capitolo 6 – Conclusioni	17

Capitolo 0 – Introduzione

Il progetto è stato sviluppato nell'ambito del corso di Ingegneria della Conoscenza presso l'Università degli Studi di Bari Aldo Moro.

L'obiettivo è la progettazione di un sistema basato su conoscenza capace di analizzare dati storici dei campionati di calcio di Serie A e di prevedere l'esito delle partite (1 = vittoria squadra casa, X = pareggio, 2 = vittoria squadra ospite) integrando tecniche di apprendimento supervisionato, ricerca euristica e ragionamento logico mediante Prolog.

Il sistema mira a fornire un supporto decisionale interpretabile e adattabile al contesto sportivo.

Dominio e requisiti funzionali

La Serie A rappresenta il massimo campionato professionistico di calcio in Italia ed è composta da venti squadre che si affrontano ogni anno in un girone di andata e ritorno.

Ogni vittoria assegna tre punti, il pareggio un punto e la sconfitta nessun punto.

Si tratta di un ambiente complesso e fortemente variabile, in cui fattori come il vantaggio casalingo, forma recente, differenza reti e/o la forza dell'avversario possono incidere in modo significativo l'esito delle partite.

Il progetto sfrutta questo contesto per costruire un sistema intelligente capace di analizzare i dati storici delle partite e di inferire, la probabilità di vittoria, pareggio o sconfitta di una squadra.

Per la realizzazione del progetto ho scelto Python per la sua ampia diffusione in ambito data science e artificial intelligence e per la disponibilità di numerose librerie specifiche per l'apprendimento automatico, l'elaborazione dei dati e il ragionamento simbolico.

Versione Python: 3.11 o superiore

IDE: PyCharm

Librerie principali:

pandas, numpy, scikit-learn, pgmpy, matplotlib, pyswip, imbalanced-learn

Struttura

Il progetto è stato strutturato in cartelle quali:

- data/ - contiene tutti i file csv utilizzati durante le fasi di addestramento e validazione del sistema;
- ml/ - contiene gli script dedicati alla parte di Machine Learning, addestramento e valutazione dei modelli predittivi; comprende funzioni di cross-validation, grid search e salvataggio dei modelli;
- kb/ - ospita la base di conoscenza in Prolog che racchiude fatti e regole per la conoscenza del dominio calcistico, e la classe PrologKB, per il caricamento dinamico dei fatti, l'invocazione delle regole e l'interrogazione tramite pyswip;
- bn/ - dedicata alla rappresentazione delle relazioni probabilistiche, come la Bayesian Network costruita per analizzare dipendenze tra variabili del dataset (es. forma squadra <--> risultato);

- search/ - modulo relativo alle tecniche di ricerca e ottimizzazione, per generare schedine, con rischio controllato, rispettando vincoli predefiniti;
- tools/ - contiene gli script per il funzionamento complessivo del sistema, predizione singola e/o multipla;
- artifacts/ - contiene i risultati prodotti dai vari moduli: modelli addestrati, risultati delle grid search, ecc.

Installazione e avvio del sistema

```
git clone https://github.com/domancler/icon.git
```

```
cd SerieA-DataLab
```

```
pip install -r requirements.txt
```

Verificare la presenza del file *from17to25.csv* dentro la cartella *data/raw/*.

Se i file di train e test non sono presenti in *data/processed/*, eseguire:

```
python -m tools.split_by_season
```

Confermata la presenza dei due file possiamo lanciare il comando di addestramento:

```
python -m ml.train_eval
```

Con l'addestramento fatto il sistema è pronto ad usare le funzioni principali.

- Predizione singola/multipla da file:

```
python -m tools.predict_ticket --date "DD/MM/YYYY" --infile "percorsoassoluto\partite.txt"
```
- Predizione singola/multipla da stdin:

```
python -m tools.predict_ticket --date "DD/MM/YYYY"
```
- Creazione di una schedina con rischio controllato da file:

```
python -m search.fixture_selector --date "20/09/2025" --infile "percorsoassoluto\partite.txt"
```
- Creazione di una schedina con rischio controllato da stdin:

```
python -m search.fixture_selector --date "20/09/2025"
```

Esempio di risultato:

```
Pisa - Lazio => 1 (forzato) (H=29.37% X=25.00% A=45.62%)
Cagliari - Sassuolo => 1 (H=46.14% X=27.11% A=26.76%)
Bologna - Torino => 1 (H=46.14% X=27.11% A=26.76%)
Genoa - Cremonese => 1 (H=46.14% X=27.11% A=26.76%)
Inter - Fiorentina => 1 (H=48.06% X=26.36% A=25.58%)
Como - Verona => 1 (H=59.30% X=21.72% A=18.98%)
Juventus - Udinese => 1 (H=53.85% X=24.19% A=21.96%)
Roma - Parma => 1 (H=54.88% X=21.73% A=23.39%)
Atalanta - Milan => 1 (H=37.20% X=28.40% A=34.40%)
Lecce - Napoli => 2 (H=31.38% X=22.15% A=46.46%)

=== Schedina ===
Probabilità schedina: 0.04%
```

Capitolo 1 – Dataset

Per la realizzazione del sistema ho utilizzato dei dataset, forniti da fonti pubbliche e open source, in particolare dal repository ufficiale di [DataHub.io](https://datahub.io), contenente i risultati storici dei campionati di calcio di Serie A relativi alle stagioni comprese tra il 2017/18 e il 2024/25.

I dati, in formato CSV, sono stati raccolti e successivamente integrati in un file unico denominato *from17to25.csv*, collocato nella directory *data/raw/*.

La scelta delle stagioni non è stata casuale ma è basata sul periodo di introduzione della tecnologia VAR (Video Assistant Referee) che ha inciso sulle corrette valutazioni da campo e conseguentemente sull'esito delle partite.

Ogni riga del dataset rappresenta una singola partita e riporta le informazioni principali relative all'incontro, come la data, le squadre coinvolte, i gol realizzati, l'esito finale ecc.

E' composto da 3030 partite e 21 colonne, quali:

Season	Indica la stagione calcistica di riferimento (es. "24/25")
Date	Data della partita in formato dd/mm/yyyy
HomeTeam	Squadra di casa
AwayTeam	Squadra ospite
FTHG	Gol segnati dalla squadra di casa
FTAG	Gol segnati dalla squadra ospite
FTR	Esito finale della partita (H = vittoria squadra casa, D = pareggio, A = vittoria squadra ospite)
HTHG	Gol segnati dalla squadra di casa al termine del primo tempo
HTAG	Gol segnati dalla squadra ospite al termine del primo tempo
HTR	Risultato al termine del primo tempo
HS	Numero di tiri totali della squadra di casa
AS	Numero di tiri totali della squadra ospite
HST	Numero di tiri in porta della squadra di casa
AST	Numero di tiri in porta della squadra ospite
HF	Falli commessi dalla squadra di casa
AF	Falli commessi dalla squadra ospite
HC	Calci d'angolo per la squadra di casa
AC	Calci d'angolo per la squadra ospite
HY	Cartellini gialli per la squadra di casa
AY	Cartellini gialli per la squadra ospite
HR	Cartellini rossi per la squadra di casa
AR	Cartellini rossi per la squadra ospite

La colonna "Season" segnata in tabella non è presente nei dataset di DataHub ma ho preferito aggiungerla per comodità di sviluppo.

Pulizia dei dati

Dopo la raccolta e l'unificazione dei dati relativi alle partite delle varie stagioni, e prima di poter procedere all'addestramento dei modelli, ho ritenuto necessario effettuare una pulizia del dataset in modo da avere una situazione più leggibile. In questa fase viene utilizzato il modulo *ml/features.py* che implementa più funzioni per elaborare il dataset.

Tra queste *normalize()* ha il compito di uniformare la struttura del dataset rendendolo più leggibile e coerente con le funzioni successive. Durante questa fase viene anche eseguita una verifica della completezza del dataset. Le date, inoltre, vengono convertite nel formato *datetime* di Python, garantendo la possibilità di effettuare ordinamenti cronologici e calcolare facilmente le sequenze temporali necessarie nelle fasi successive.

Elaborazione delle feature

Verificato e pulito il dataset, ho aggiunto informazioni derivate, che mi torneranno utili successivamente, capaci di rappresentare aspetti che non emergono dai dati grezzi. Questa fase viene implementata tramite funzioni di *ml/features.py*.

Tra di queste, la più rilevante è la *rolling_form()*. Questa funzione calcola lo stato di forma recente di ogni squadra, prendendo le statistiche delle ultime cinque partite (antecedenti alla data richiesta), calcolandosi per la squadra di casa e ospite:

- punti totalizzati
- goal fatti
- goal subiti
- differenza goal
- numero di vittorie
- numero di pareggi
- numero di sconfitte

Sempre nel modulo *ml/features.py* è presente la funzione *build_matrix()*, che crea e sistema la matrice contenente squadre, esito, forma e statistiche che sarà successivamente usata per l'apprendimento.

Capitolo 2 – Apprendimento Supervisionato

L'apprendimento supervisionato è una tecnica di machine learning basata sul fornire al modello un insieme di dati etichettati e far apprendergli la relazione tra features e labels.

L'obiettivo è quello di costruire un modello capace di prevedere in base a dati mai visti, sfruttando una relazione tra gli input e gli output; in questo caso, prevedere l'esito di una partita di Serie A, vittoria della squadra di casa, pareggio o vittoria della squadra ospite.

Il problema viene formalizzato come una classificazione multiclasse a tre etichette (H, D, A), in cui ciascun esempio corrisponde a una partita e ciascuna feature descrive uno o più aspetti del comportamento recente delle due squadre.

Tramite la funzione *split_by_season()* divido il dataset in modo cronologico, utilizzando come training set le stagioni dal 2017/2018 al 2023/2024 e come test set la stagione 2024/2025, al fine di garantire che il modello impari solo da dati passati e venga valutato su partite effettivamente "future".

Scelta dei modelli

I modelli che ho scelto sono stati:

- **Logistic Regression:** classificatore lineare che stima la probabilità di ciascun esito in funzione di una combinazione lineare delle feature di input. La sua principale caratteristica è l'elevata interpretabilità: i coefficienti appresi dal modello consentono di valutare il peso relativo di ciascuna variabile nel determinare un esito, offrendo una visione trasparente delle dinamiche apprese.
- **Support Vector Machine:** scelto la sua capacità di separare classi anche in spazi non linearmente separabili. Attraverso l'uso di funzioni kernel, come il radial basis function (RBF), il modello riesce a mappare i dati in spazi a dimensionalità superiore, trovando iperpiani che massimizzano il margine di separazione tra le classi. In questo contesto la SVM consente "catturare" effetti non lineari che la regressione logistica non sarebbe in grado di rappresentare.
- **Random Forest:** un modello di tipo ensemble che combina i risultati di molteplici alberi di decisione. Ogni albero viene addestrato su un sottoinsieme casuale dei dati e delle feature, e il risultato finale è ottenuto tramite voto maggioritario o media delle probabilità. Questo approccio riduce la varianza tipica dei singoli alberi e garantisce un buon compromesso tra accuratezza e robustezza.

La combinazione di questi tre modelli permette al sistema di valutare e confrontare approcci diversi — lineare, non lineare e basato su ensemble — ottenendo una visione complessiva delle potenzialità predittive del dataset.

Pipeline di addestramento e validazione

L'intero processo di addestramento è gestito dal modulo `ml/train_eval.py`, che si occupa di automatizzare la costruzione, la validazione e la valutazione dei modelli. Questo file contiene le funzioni principali `train_models()` ed `evaluate_model()`, che implementano una pipeline di addestramento coerente con le buone pratiche di machine learning supervisionato.

La pipeline inizia con il caricamento dei dataset di training e test, seguita dalla normalizzazione delle feature numeriche tramite `StandardScaler`, operazione necessaria per evitare che le variabili con valori più grandi dominino la funzione obiettivo.

```
return {
    "logreg": make_pipeline(
        *steps: StandardScaler(with_mean=False),
        LogisticRegression(solver="lbfgs")
    ), # buon solver generico e stabile
    "svm": make_pipeline(
        *steps: StandardScaler(with_mean=False),
        SVC(probability=True)
    ),
    "rf": RandomForestClassifier(random_state=42)
}
```

Successivamente, ogni modello viene addestrato su più fold del dataset attraverso la tecnica della K-Fold Cross Validation. In questo modo, le prestazioni vengono stimate non su una singola suddivisione dei dati, ma su più partizioni, riducendo il rischio di overfitting e garantendo una valutazione più stabile e affidabile.

Per ciascun modello vengono inoltre eseguite procedure di Grid Search sugli iperparametri più sensibili, come il parametro di regolarizzazione C per la SVM o la profondità massima degli alberi nella Random Forest.

La griglia di ricerca consente di identificare automaticamente la combinazione di parametri che massimizza le prestazioni, misurate tramite accuracy e log-loss. Al termine della validazione, i modelli ottimizzati vengono salvati nella cartella `artifacts/`.

Ottimizzazione degli iperparametri

Uno degli aspetti più delicati nella costruzione di un modello di apprendimento supervisionato riguarda la scelta dei iperparametri, quei parametri esterni che controllano il comportamento del modello ma non vengono appresi direttamente dai dati. Valori troppo restrittivi possono portare a modelli sottotratati (underfitting), mentre una complessità eccessiva rischia di produrre sovradattamento (overfitting).

Per la ricerca di iperparametri ottimali ho usato la tecnica di Grid Search. Questa procedura consiste nel testare automaticamente diverse combinazioni di valori predefiniti e nel valutare le prestazioni del modello su ciascun insieme di parametri.

```
SPACE = {
  "logreg": {
    "logisticregression__C": [0.1,1,3,10],
    "logisticregression__max_iter": [300]
  },
  "svm": {
    "svc__C": [0.5,1,3],
    "svc__kernel": ["linear", "rbf"]
  },
  "rf": {
    "n_estimators": [200,400,800],
    "max_depth": [None,10,20],
    "min_samples_split": [2,5]
  }
}
```

Per la Regressione Logistica, la ricerca si è concentrata principalmente sul parametro di regolarizzazione C, che controlla l'intensità della penalizzazione sui coefficienti del modello: valori più piccoli di C (es. 0.1) impongono una regolarizzazione più forte, riducendo la varianza ma aumentando il bias, mentre valori più grandi (es. 10) rendono il modello più flessibile, ma anche più soggetto a oscillazioni dovute al rumore dei dati.

Nel caso della SVM, gli iperparametri principali esplorati sono stati il parametro C, che regola il compromesso tra errore di classificazione e ampiezza del margine, e il parametro gamma, che definisce l'influenza dei singoli punti nel kernel RBF. La ricerca ha permesso di individuare il bilanciamento migliore tra i due parametri, ottimizzando la capacità del modello di distinguere le classi anche in regioni non linearmente separabili.

Per la Random Forest, gli iperparametri analizzati includono il numero di alberi (`n_estimators=[200,400,800]`), la profondità massima (`max_depth=[None, 10, 20]`) e la dimensione minima dei nodi foglia (`min_samples_leaf=[2,5]`).

L'obiettivo, in questo caso, è stato quello di controllare la complessità complessiva dell'ensemble per evitare che il modello si adatti eccessivamente ai dati di training, mantenendo però una buona capacità discriminante.

Al termine della ricerca, la configurazione ottimale è stata scelta sulla base della cross-validation accuracy media e della stabilità dei risultati nei diversi fold.

Modello	Parametri ottimali	Accuracy media (CV)	Dev. std (CV)
LogisticRegression	C=10, max_iter=300	0.4930	0.0086
RandomForestClassifier	max_depth=10, min_samples_split=5, n_estimators=800	0.4664	0.0220
SVC	C=0.5, kernel=linear	0.4939	0.0099

Valutazione e confronto dei modelli

Una volta completata la fase di addestramento e tuning, i modelli sono stati valutati sul set di test, corrispondente alla stagione 2024/2025, al fine di stimare la loro capacità di predire risultati mai osservati durante l'apprendimento. La valutazione è stata condotta utilizzando più metriche, così da ottenere una visione complessiva delle prestazioni e non dipendere da un unico indicatore.

La accuracy è stata utilizzata come metrica principale, poiché misura la percentuale di partite correttamente predette rispetto al totale.

Tuttavia, considerando la leggera prevalenza di vittorie casalinghe, sono stati calcolati anche la precisione, il recall e la F1-score per ciascuna classe, in modo da valutare l'efficacia del modello anche nei confronti di esiti meno frequenti come il pareggio o la vittoria esterna.

È stata inoltre analizzata la log-loss, che considera la qualità probabilistica delle predizioni, penalizzando con maggiore severità le previsioni errate ma molto sicure.

I risultati medi ottenuti nei vari run di validazione hanno evidenziato un comportamento coerente con le caratteristiche dei modelli:

- la Regressione Logistica ha garantito buone prestazioni di base e una chiara interpretabilità, confermandosi come baseline affidabile;
- la SVM ha mostrato un miglioramento in termini di accuratezza grazie alla capacità di modellare relazioni non lineari, soprattutto nelle partite con esiti incerti;
- la Random Forest, infine, ha raggiunto il miglior equilibrio tra accuratezza complessiva e stabilità, risultando il modello più robusto alle variazioni stagionali.

Conclusione

La Random Forest si è rivelata la scelta più solida, combinando affidabilità e capacità di generalizzazione, ma l'intero processo di sperimentazione ha evidenziato l'importanza di una pipeline rigorosa e sistematica per la validazione dei risultati.

Capitolo 3 – Rete Bayesiana

In questo contesto la rete bayesiana rappresenta la componente assegnata alla gestione dell'incertezza e alla modellazione delle relazioni probabilistiche tra le variabili che descrivono le prestazioni delle squadre.

Mentre i modelli di machine learning supervisionato si basano su correlazioni statistiche apprese direttamente dai dati, la rete bayesiana consente di formalizzare dipendenze causali esplicite, offrendo una rappresentazione più interpretabile del processo decisionale.

Nel dominio calcistico, le relazioni tra le variabili raramente sono deterministiche.

Ad esempio, una squadra in buona forma (high form) non vince sempre, ma ha una probabilità più alta di farlo; allo stesso modo, una differenza reti favorevole o un vantaggio in termini di tiri in porta possono aumentare la probabilità di vittoria, ma non la garantiscono.

La rete bayesiana nasce proprio per catturare queste relazioni di tipo probabilistico condizionato, fornendo una struttura che unisce conoscenza qualitativa e dati empirici.

L'uso della rete bayesiana qui ha quindi un duplice obiettivo:

- Integrare la conoscenza appresa dai modelli supervisionati con una rappresentazione probabilistica più interpretabile e flessibile.
- Fornire una base di inferenza autonoma, in grado di stimare la probabilità di ciascun esito (1, X, 2) anche in assenza di un modello predittivo complesso.

Costruzione della rete

Ho implementato la rete bayesiana nel modulo *bn/bayes_model.py* mediante l'utilizzo della libreria pgmpy, una delle più diffuse in ambito Python per la modellazione di reti probabilistiche grafiche.

La struttura della rete è costituita da un insieme di nodi, che rappresentano variabili osservabili o derivate dal dataset, e da archi diretti, che descrivono le relazioni di dipendenza condizionata tra tali variabili.

Ho progettato la rete in modo che rappresenti i principali fattori che influenzano l'esito di una partita, con una struttura volutamente semplice ma concettualmente chiara.

I nodi principali sono i seguenti:

- *H_form*: rappresenta la forma recente della squadra di casa, calcolata come somma dei punti ottenuti nelle ultime cinque partite (0–15).
- *A_form*: forma recente della squadra ospite, calcolata nello stesso modo.
- *H_gd*: differenza reti della squadra di casa nelle ultime cinque partite.
- *A_gd*: differenza reti della squadra ospite nelle ultime cinque partite.
- *Result*: variabile target che indica l'esito finale della partita (H, D, A).

Le relazioni causali sono state definite in modo da riflettere la logica calcistica di base: le prestazioni e le caratteristiche delle squadre (forma e differenza reti) influenzano l'esito della partita, ma non viceversa.

La rete viene addestrata tramite la funzione *train_bn()*, che stima le tabelle di probabilità condizionate (CPT) a partire dai dati storici. Per farlo, faccio prima elaborare i dati tramite la funzione *build_bn_dataframe()*, che converte le variabili numeriche in categorie discrete — ad esempio, la forma di una squadra viene suddivisa in livelli (“bassa”, “media”, “alta”) sulla base di soglie predefinite.

La struttura così definita non pretende di rappresentare tutte le complessità del gioco del calcio, ma fornisce una base probabilistica compatta e interpretabile, sufficiente per cogliere le dipendenze fondamentali tra rendimento e risultato.

Addestramento e inferenza

Il processo di addestramento della rete bayesiana avviene in due fasi principali: la preparazione dei dati e la stima delle distribuzioni condizionate.

Dopo aver costruito il dataset con *build_bn_dataframe()*, la funzione *train_bn()* utilizza i dati discreti per apprendere le probabilità a priori e le tabelle di probabilità condizionate di ciascun nodo.

Una volta addestrata, la rete viene utilizzata per eseguire inferenze tramite la funzione *predict_bn()*, che riceve in input le evidenze relative alle squadre coinvolte in una partita. Ad esempio, se la squadra di casa è in buona forma e presenta una differenza reti positiva, la rete può stimare la probabilità condizionata di vittoria:

$P(\text{Result} = H \mid H_form = \text{high}, A_form = \text{mid}, H_gd = \text{pos}, A_gd = \text{eq})$

Il risultato di questa inferenza è un dizionario contenente le probabilità di ciascun esito, ad esempio:

`{'H': 0.48, 'D': 0.29, 'A': 0.23}`

Queste probabilità rappresentano una stima probabilistica ragionata dell'esito, che tiene conto non solo dei dati osservati, ma anche delle relazioni nella rete.

A differenza di un modello supervisionato, la rete bayesiana non richiede un addestramento esteso o una grande quantità di dati per fornire stime significative: è in grado di inferire anche in condizioni di informazione parziale, ossia quando alcune evidenze non sono disponibili.

Integrazione con il sistema

La rete bayesiana, una volta addestrata, non opera in modo isolato ma viene integrata all'interno del flusso principale del sistema, collaborando con i modelli di apprendimento supervisionato e con la base di conoscenza logica in Prolog.

Durante la predizione di una o più partita (schedina), il sistema procede secondo la seguente sequenza:

1. Predizione supervisionata – viene caricato il modello di Machine Learning ottimizzato (ad esempio la Random Forest) e utilizzato per stimare le probabilità di ciascun esito in base alle feature numeriche calcolate. Il risultato di questa fase è un vettore di probabilità empiriche:
`p_ml = {'H': 0.52, 'D': 0.28, 'A': 0.20}`.
2. Inferenza bayesiana – viene costruita una rete bayesiana aggiornata sulla base delle ultime cinque partite di entrambe le squadre, attraverso la funzione `predict_bn()`. La rete restituisce un nuovo vettore di probabilità, derivato dalla combinazione delle evidenze discreti:
`p_bn = {'H': 0.45, 'D': 0.35, 'A': 0.20}`.
3. Inferenza logica (Prolog) – la base di conoscenza (`kb/knowledge.pl`), gestita tramite la classe `PrologKB`, fornisce una terza fonte informativa, non più statistica ma simbolica. Essa applica regole del tipo:
`likely_draw(Home, Away) :- is_derby(Home, Away).`
`likely_home(Home, Away) :- strong_form(Home).`

e genera una distribuzione qualitativa: `p_pl = {'H': 0.4, 'D': 0.4, 'A': 0.2}`.

A questo punto, i tre vettori vengono fusi mediante una funzione di combinazione pesata, come `fuse3(p_pl, p_bn, p_ml, w_pl, w_bn)`, che assegna un peso a ciascuna componente.

Il valore finale rappresenta la probabilità complessiva dell'esito, calcolata come media ponderata tra i tre contributi:

```
w_ml = max(0.0, 1.0 - (w_pl + w_bn))
out = {k: w_pl * (p_pl.get(k, 0)) + w_bn * (p_bn.get(k, 0)) + w_ml * (p_ml.get(k, 0)) for k in keys}
s = sum(out.values())
return {k: (v/s if s>0 else 0.0) for k,v in out.items()}
```

La fusione di questi modelli consente di sfruttare le rispettive caratteristiche: la rete bayesiana introduce una visione probabilistica strutturata e coerente con le dipendenze causali; i modelli supervisionati forniscono la capacità di apprendere pattern complessi dai dati numerici; la base logica aggiunge interpretabilità e regole qualitative difficilmente apprendibili da soli dati statistici.

In questo modo, la rete bayesiana diventa una sorta di “ponte semantico” tra la componente di apprendimento empirico e quella di ragionamento simbolico, assicurando che le predizioni finali tengano conto sia dell’evidenza numerica sia della conoscenza di dominio.

Discussione e limiti

La semplicità strutturale della rete implementata rappresenta anche il suo principale limite. Ho scelto di considerare poche relazioni (forma e differenza reti) per mantenere la rete leggibile e facilmente addestrabile, ma ciò comporta una riduzione della granularità nella rappresentazione del dominio.

Un’estensione futura potrebbe prevedere l’introduzione di ulteriori nodi, come “posizione in classifica”, “tiri in porta medi” o “probabilità di gol atteso (xG)”, al fine di rendere il modello più espressivo.

Capitolo 4 – Rappresentazione della conoscenza e Prolog

Mentre i modelli di machine learning e la rete bayesiana si basano su correlazioni e probabilità, la componente Prolog permette di esprimere conoscenza strutturata in forma di regole e fatti, rendendo possibile l’inferenza logica e la generazione di nuova conoscenza a partire da informazioni esistenti.

Questo approccio si ispira ai principi classici dei Sistemi Basati su Conoscenza (KBS): la separazione tra la base di conoscenza (che descrive il dominio) e il motore di inferenza (che elabora le regole per derivare nuove informazioni).

In questo contesto, questa architettura è stata realizzata attraverso due componenti principali:

- Il file *kb/knowledge.pl*, che contiene i fatti e le regole logiche relative al dominio calcistico.
- Il modulo *kb/prolog_engine.py*, che funge da interfaccia tra Python e Prolog, consentendo al sistema di interrogare la base di conoscenza e aggiornare dinamicamente i fatti in memoria.

L’obiettivo non è sostituire il ragionamento statistico, ma completarlo.

Con l’uso di Prolog, non mi limito a stimare la probabilità di un esito, ma posso anche motivarlo logicamente — ad esempio, riconoscendo che un determinato incontro è un derby o che una squadra è in stato di forma positiva.

La base di conoscenza (knowledge.pl)

Nel file *kb/knowledge.pl* vengono definiti i fatti dinamici e le regole di inferenza che descrivono il dominio calcistico in termini logici. Ogni regola rappresenta una relazione tra entità, e ogni fatto descrive una condizione osservabile o calcolata nel sistema.

I fatti dinamici rappresentano lo stato corrente delle squadre e delle partite analizzate. Essi vengono dichiarati e aggiornati automaticamente dal motore Python attraverso la classe PrologKB.

Tra i principali fatti dinamici troviamo:

```
points_last5(Team, Points).  
gd_last5(Team, GD).  
match(Date, Home, Away).
```

Questi fatti indicano rispettivamente i punti ottenuti da una squadra nelle ultime cinque partite, la differenza reti nello stesso periodo e la partita correntemente analizzata.

Essi vengono continuamente rimossi tramite `_clean_dynamic_facts()` e ricreati tramite `assert_match_facts()` per ogni predizione, così da mantenere la base di conoscenza coerente e aggiornata.

Regole di inferenza

Le regole logiche descrivono relazioni di alto livello tra le variabili e permettono al sistema di dedurre proprietà implicite.

Ad esempio:

`likely_draw(Home, Away) :- is_derby(Home, Away).`

`likely_home(Home, Away) :- high_gap(Home, Away).`

`likely_away(Home, Away) :- strong_form(Away).`

In queste regole, il predicato `likely_draw/2` indica che, se due squadre appartengono alla stessa città (quindi è un derby), l'esito più probabile è il pareggio.

Analogamente, `likely_home/2` e `likely_away/2` rappresentano condizioni di predominio statistico o prestazionale.

Queste regole non producono una predizione numerica, ma un'inferenza qualitativa che può essere tradotta in una distribuzione logica di probabilità e poi integrata con le stime della rete bayesiana e dei modelli di machine learning.

Inoltre, la base contiene regole di supporto come `strong_form/1` o `high_gap/2`, che formalizzano concetti calcistici in forma deduttiva, ad esempio:

`strong_form(T) :- points_last5(T, P), P >= 8.`

`high_gap(Home, Away) :- gd_last5(Home, GDH), gd_last5(Away, GDA), Gap is GDH - GDA, Gap >= 6.`

Quindi una squadra con almeno 8 punti nelle ultime 5 partite è considerata "in forma", mentre una differenza reti superiore a 6 tra due squadre suggerisce un vantaggio significativo.

Si tratta di conoscenza esplicita, formalizzata in modo dichiarativo e facilmente estendibile, che costituisce la componente semantica del sistema.

Inferenza logica

Una volta popolata la base di conoscenza, il metodo `query_likely()` interroga i predicati principali (`likely_home`, `likely_draw`, `likely_away`) per verificare quali regole risultano vere nel contesto corrente.

I risultati vengono poi elaborati da `logical_prior()`, che restituisce un dizionario con le probabilità qualitative:

```
{'H': 0.4, 'D': 0.4, 'A': 0.2}
```

In questo modo, l'inferenza logica diventa compatibile con la componente probabilistica del sistema, pur mantenendo una natura simbolica.

Spiegazioni qualitative

Il metodo `explain()` consente di interrogare la base di conoscenza per comprendere perché una determinata inferenza è stata prodotta.

Ad esempio, se due squadre appartengono alla stessa città, la funzione restituirà “derby $\Rightarrow$ draw”; se una squadra ha ottenuto più di 8 punti nelle ultime 5 partite, l’output sarà “strong_form(home) $\Rightarrow$ home”.

Questo meccanismo di spiegazione rende il sistema trasparente e interpretabile, una caratteristica rara nei modelli puramente numerici.

Prolog

All’interno dell’architettura generale di SerieA-DataLab, Prolog rappresenta la componente dedicata al ragionamento simbolico e all’integrazione della conoscenza di dominio.

La base di conoscenza non sostituisce i modelli predittivi, ma fornisce un contributo qualitativo che viene combinato con gli output delle altre componenti nel modulo *tools/predict_ticket.py*.

Durante la generazione di una schedina o la predizione di una partita, il sistema:

- crea un’istanza di PrologKB;
- aggiorna i fatti relativi alle squadre coinvolte;
- interroga la base per ottenere una stima logica degli esiti più plausibili;
- fonde queste informazioni con le probabilità provenienti dal modello bayesiano e dal classificatore supervisionato.

In questo modo, Prolog contribuisce a rafforzare la coerenza semantica del sistema, aggiungendo alle pure probabilità numeriche una dimensione di ragionamento esplicito.

Ad esempio, un pareggio previsto in un derby non risulta più soltanto da una correlazione statistica, ma anche da una regola logica coerente con la conoscenza calcistica incorporata nella KB.

Considerazioni finali

La base Prolog, seppur semplice nella sua forma attuale, offre un linguaggio espressivo e dichiarativo, facilmente estendibile con nuove regole, condizioni o predicati.

In prospettiva, si potrebbe migliorare la sua conoscenza con l’aggiunta di nuove informazioni come ad esempio informazioni tattiche, dati sugli infortuni o prestazioni per ruolo ecc. Questo potrebbe rendere il sistema ancora più flessibile e vicino al funzionamento del ragionamento umano nel dominio sportivo.

Capitolo 5 – Ricerca euristica e schedina rc

Sviluppato i modelli di predizione e la base di conoscenza, il passo successivo è stato progettare un meccanismo che permettesse al sistema di utilizzare le previsioni generate per comporre non solo schedine che riportassero l’esito più probabile (poiché spesso è 1=H o 2=A), ma una schedina con un rischio controllato, ovvero, una schedina che introduce una predizione non solo “probabilistica” ma una predizione con un rischio controllato (rc). Questo può comportare una schedina più “audace” in quanto forza

l'inserimento di pareggi ma allo stesso tempo riduce altri rischi come l'inserimento di più partite troppo equilibrate.

Questa funzionalità, racchiusa nel modulo *search/fixture_selector.py*, non si limita a scegliere partite a caso, ma impiega un algoritmo di ricerca euristica per massimizzare la qualità complessiva delle previsioni, rispettando allo stesso tempo alcuni vincoli imposti dall'utente (ad esempio, numero minimo di pareggi o massimo di partite "troppo equilibrate").

Il problema può essere visto come una forma di ottimizzazione combinatoria: tra tutte le possibili combinazioni di partite e risultati previsti, si desidera trovare quella che massimizza una funzione di punteggio, bilanciando affidabilità e rischio.

Poiché il numero di combinazioni cresce in modo esponenziale con il numero di partite, un approccio esaustivo risulterebbe impraticabile.

Per questo motivo, si è scelto di adottare una strategia euristica basata su Hill Climbing, che consente di esplorare in maniera efficiente lo spazio delle soluzioni.

Hill Climbing

L'algoritmo Hill Climbing è una tecnica di ricerca locale che parte da una soluzione iniziale casuale e tenta di migliorarla iterativamente, sostituendo elementi della soluzione corrente con alternative potenzialmente migliori.

A ogni iterazione, viene generata una nuova combinazione (una "soluzione vicina") e valutata tramite una funzione di punteggio.

Se la nuova soluzione è migliore della precedente, viene accettata e diventa la base per l'iterazione successiva; altrimenti, viene scartata.

In questo contestop, ogni soluzione corrisponde a un insieme di K partite selezionate da un turno di campionato, ciascuna con un esito previsto (1, X, 2).

La funzione di punteggio, definita all'interno di *fixture_selector.py*, valuta ogni schedina in base a due criteri principali:

- Affidabilità complessiva – rappresentata dalla media delle probabilità più alte tra i tre esiti possibili per ciascuna partita (ricavate dalle predizioni ML e BN). In altre parole, se una partita ha una previsione fortemente sbilanciata (es. 0.75–0.15–0.10), essa contribuisce positivamente al punteggio.
- Rispetto dei vincoli – ogni schedina deve rispettare condizioni predefinite, come:
 - un numero minimo di pareggi (`min_draw`);
 - un limite massimo di partite molto equilibrate e quindi rischiose (`max_low_margin`);

Le schedine che non rispettano tali vincoli vengono scartate o penalizzate.

Formalmente, la funzione di valutazione può essere espressa come:

$$\text{score}(S) = \begin{cases} \frac{1}{K} \sum_{i=1}^K p_i & \text{se i vincoli sono rispettati} \\ -1 & \text{altrimenti} \end{cases}$$

dove p_i rappresenta la probabilità massima associata alla partita i-esima.

Il processo inizia con una schedina casuale e procede per un numero definito di iterazioni (iters, tipicamente 1000), sostituendo a ogni passo una partita della schedina con un'altra non ancora inclusa.

Il risultato finale è la combinazione con il punteggio più alto, cioè la schedina euristicamente ottimizzata in base alle condizioni impostate.

Parametri e funzionamento del modulo

Il file *fixture_selector.py* ha come funzione principale `hill_climb(matches, K, min_draw, iters, max_low_margin, rand_seed)`, che implementa la logica descritta.

Il parametro `matches` contiene la lista di partite con le relative probabilità predette (pH, pD, pA), mentre gli altri parametri controllano il comportamento della ricerca.

Durante l'esecuzione, il sistema:

1. genera una schedina iniziale casuale di K partite;
2. calcola il punteggio iniziale con la funzione di valutazione;
3. ripete fino a `iters` iterazioni i seguenti passi:
4. seleziona casualmente una partita dalla schedina corrente da rimuovere;
5. seleziona una nuova partita da inserire tra quelle non ancora scelte;
6. ricalcola il punteggio della nuova schedina;
7. se la nuova combinazione è migliore, la sostituisce alla precedente.

Il risultato viene restituito sotto forma di elenco ordinato, dove per ciascuna partita è indicato l'esito più probabile e la relativa probabilità, ad esempio:

```
Schedina con rischio controllato:  
Atalanta - Milan => X (H=37.20% X=28.40% A=34.40%)  
Como - Verona => 1 (H=59.30% X=21.72% A=18.98%)  
Roma - Parma => 1 (H=54.88% X=21.73% A=23.39%)  
Juventus - Udinese => 1 (H=53.85% X=24.19% A=21.96%)  
Inter - Fiorentina => 1 (H=48.06% X=26.36% A=25.58%)  
Score: (48.8962%, 24.6430%, 1)
```

Oltre alla schedina finale, la funzione restituisce anche il punteggio complessivo, che misura la qualità media della combinazione individuata.

La scelta di un approccio di tipo Hill Climbing deriva da un compromesso tra semplicità ed efficacia. A differenza delle strategie di ricerca esaustiva, che avrebbero un costo computazionale proibitivo, l'Hill Climbing consente di ottenere risultati soddisfacenti in tempi brevi, esplorando solo una porzione significativa dello spazio delle soluzioni.

Inoltre, la presenza dei vincoli consente di orientare la ricerca verso combinazioni realisticamente plausibili, evitando schedine eccessivamente sbilanciate o irrealistiche (ad esempio con troppi pareggi).

Considerazioni e sviluppi futuri

Nonostante la semplicità apparente, il modulo di ricerca euristica rappresenta un elemento chiave del progetto, poiché consente di tradurre la conoscenza appresa in un output tangibile e utile.

L'approccio Hill Climbing si è dimostrato efficace per un numero limitato di partite, ma potrebbe essere ulteriormente migliorato attraverso:

- strategie di multi-start hill climbing, per esplorare più soluzioni iniziali indipendenti;
- metodi di simulated annealing o tabu search, che permetterebbero di sfuggire ai minimi locali;

Capitolo 6 – Conclusioni

In questo progetto è stato sviluppato un sistema capace di analizzare dati storici dei campionati di Serie A e di prevedere gli esiti delle partite attraverso modelli di apprendimento supervisionato e tecniche di ricerca euristica.

Sono stati sperimentati diversi classificatori — Logistic Regression, SVM e Random Forest — ottimizzati mediante Grid Search e cross-validation; successivamente, le previsioni sono state utilizzate per generare schedine tramite l'algoritmo Hill Climbing, in grado di bilanciare l'affidabilità delle scelte con un livello di rischio controllato.

Dai risultati ottenuti emerge come sia possibile modellare alcuni pattern ricorrenti nel campionato, considerando fattori come la forma recente, il vantaggio casalingo e la differenza reti.

Tra gli sviluppi futuri possibili vi è l'ampliamento del dataset a più competizioni, l'integrazione di nuovi modelli probabilistici per un confronto più ampio e il potenziamento della base di conoscenza logica in Prolog per introdurre regole strategiche e condizioni dinamiche.

Un'ulteriore evoluzione potrà riguardare la definizione di metriche di rischio personalizzabili e l'inclusione di ulteriori variabili descrittive — come dati disciplinari (cartellini), quotazioni pre-partita o informazioni su assenze e infortuni — così da rendere il sistema più flessibile, realistico e aderente al contesto sportivo reale.